

Lecture 1 Notes :

System Design

System Design is the process of planning **how an entire software system will work** before building it.

Skills Needed for System Design

Topic	Category	Why
Programming Fundamentals	Foundation	You need to know how to write code before learning design.
Object-Oriented Programming (OOP)	LLD	LLD is built around classes, objects, and relationships.
Data Structures & Algorithms (DSA)	DSA	Focuses on writing efficient algorithms and choosing the right data structures.
Design Patterns	LLD	Patterns like Singleton, Factory, Observer, and Strategy help design maintainable code.
Databases (SQL & NoSQL)	HLD (mainly)	Choosing the right database and designing data storage is part of system architecture.
Networking Basics (HTTP, APIs, Client-Server)	HLD	Understanding how different services communicate is essential for system architecture.
System Design	HLD + LLD + DSA	Combines architecture (HLD) with detailed component design (LLD + DSA).

Illustrative Story: Two Approaches to Building “QuickRide”

- Scenario: Build a ride-booking app (“QuickRide”) like Uber/Pathao

A’s DSA-First Approach:

1. Problem decomposition:

- Map city intersections to graph nodes, roads to edges.
- Use algorithm to compute shortest path
- Use a priority queue to match riders to closest drivers.

2. Gaps:

- No identification of classes/entities (User, Rider, Location, Notification, Payment).
- Omits data security (masking phone numbers).
- Missing integration points (notifications, payment gateways).
- No consideration for scaling to millions of users.

B’s LLD-First Approach:

1. Entity identification:

- Objects: User, Rider, Location, NotificationService, PaymentGateway, etc.

2. Define relationships & interactions:

- How User and Rider connect via Location.
- How NotificationService and PaymentGateway integration.

3. Non-functional concerns:

- Data security: Protect personal info.
- Scalability: Architect code to handle millions of users without performance collapse.

4. Then apply DSA:

- Embed shortest-path algorithm and driver-matching, inside this object-oriented framework.

What is Low-Level Design (LLD)?

Definition: Designing the internal structure (“skeleton”) of an application by

identifying classes/objects, their relationships, data flows, and how DSA solutions plug into this structure.

- DSA: Solves isolated problems (e.g. “find shortest path in an array/graph”) using algorithms like binary search, sort, queue, etc.
- LLD: Determines which objects exist in the system and how they interact, then applies DSA inside that structure.

Core LLD Principles & Focus Areas

1. Scalability

- Handle large user volumes easily.
- Code structure should allow rapid, low-effort expansion (adding servers, features).

2. Maintainability

- New features shouldn't break existing ones.
- Code should be easy to debug and locate bugs.

3. Reusability

- Write loosely coupled, “plug-and-play” modules (e.g. generic notification or matching algorithms usable across apps like Foodpanda, Uber , Amazon delivery).

What is HLD?

- High-Level Design (HLD) focuses on system architecture, not code structure:
- Tech stack: Choice of languages/frameworks (e.g. Java Spring Boot).
- Database: SQL vs. NoSQL vs. hybrid.
- Server scaling & deployment: Autoscaling, load balancers, cost optimization on AWS/GCP.
- Cost considerations: Minimizing cloud/server expenses per load.

5. Summary & Takeaways

- DSA = Brain of an application: algorithms solve specific tasks.
- LLD = Skeleton: object models, class diagrams, code organization, and where algorithms plug in.
- HLD = Architecture: system-wide infrastructure, tech stack, databases, servers.

6. Key line to remember

“If DSA is the brain, LLD is the skeleton of your application.”